Cyprus University of Technology

CSE 226
Algorithms and Complexity

Lecture 14

Single-Source Shortest Paths (Part 1)

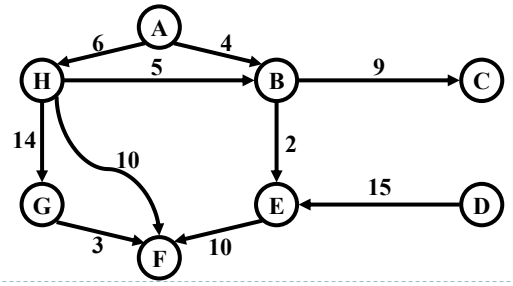
Ομοαφετηριακές Ελαφρύτατες Διαδρομές (Μέρος 1)

Herodotos Herodotou

Contains edited slides from T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, D. Luebke, and M. Christodoulakis

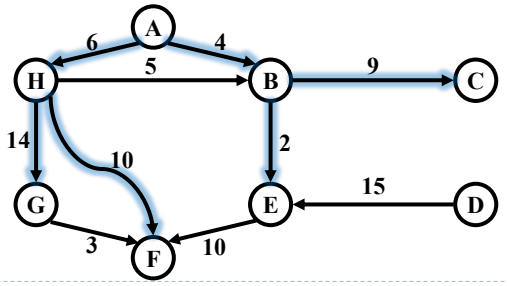
Single-Source Shortest Path

- **Problem:** Given a weighted directed graph G with a weight function $w: \mathrm{E} \to \mathbb{R}$, find the *minimum-weight path* from a given source vertex s to another vertex v
  - Weight of path $p = \langle v_0, v_1, \ldots, v_k \rangle$ is the sum of edges: $w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$
  - **Shortest path** = path with minimum weight $\boldsymbol{\delta(s, v)}$



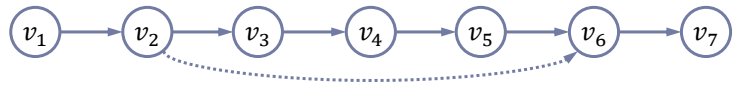
Single-Source Shortest Path

- **Problem:** Given a weighted directed graph G with a weight function $w: \mathrm{E} \to \mathbb{R}$, find the *minimum-weight path* from a given source vertex s to another vertex v
  - Weight of path $p = \langle v_0, v_1, \ldots, v_k \rangle$ is the sum of edges: $w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$
  - **Shortest path** = path with minimum weight $\boldsymbol{\delta(s, v)}$



Optimal Substructure of Shortest Path

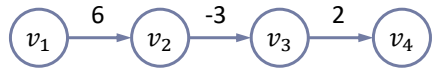
- Once again, we have **optimal substructure** (**βέλτιστη υποδομή**): the shortest path consists of shortest subpaths



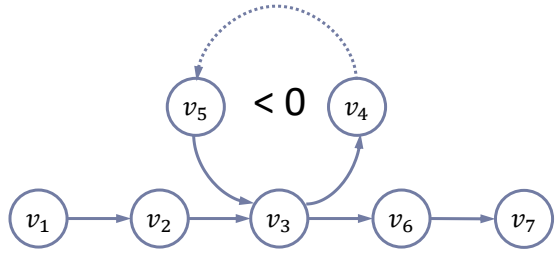
- **Proof by contradiction:**
  - Suppose some subpath is not a shortest path
  - There must then exist a shorter subpath
  - Could substitute the subpath for a shorter path
  - But then, overall path becomes shorter. Contradiction

Negative Weights

- Is it OK to have negative-weight **edges**? Yes



- Is it OK to have negative-weight **cycles**? No



Cycles

- Shortest paths **cannot** contain cycles:
  - Already ruled out negative-weight cycles
  - Positive-weight ⇒ we can get a shorter path by omitting the cycle
  - Zero-weight: no reason to use them so don't
- What is the max number of edges in a shortest path?
  - $V - 1$

Output of SSSP Algorithms

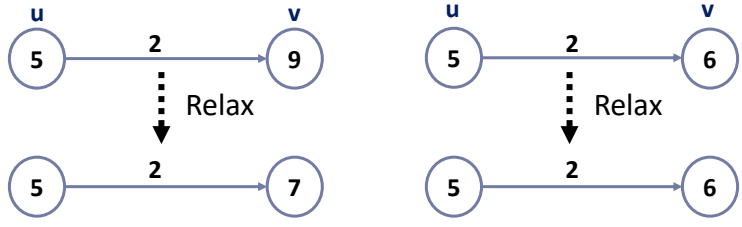
For each vertex $v \in V$:

- $d[v] = \delta(s, v)$
  - Initially, $d[v] = \infty$
  - Decreases as algorithm progresses but always maintains $d[v] \geq \delta(s, v)$
  - Call $d[v]$ a **shortest-path estimate** (**εκτίμηση ελαφρύτατης διαδρομής**)
- $p[v] =$ **predecessor** (**προκάτοχος**) of v on a shortest path from s
  - If no predecessor, $p[v] = \text{NULL}$
  - p induces a tree: **shortest-path tree** (**δέντρο ελαφρύτατων διαδρομών**)

Relaxation

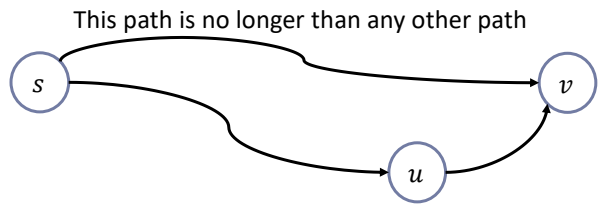
- **Relaxation** (**Χαλάρωση**)**:** Can we improve the shortest-path estimate for v by going through u and taking (u, v)?

```
Relax(u, v, w) {
   if (d[v] > d[u] + w(u,v)) then
      d[v] = d[u] + w(u,v);
      p[v] = u
}
```



Shortest Path Properties

- **Triangle inequality** (**Τριγωνική ανισότητα**):
  - For all $(u, v) \in E$, we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$



Shortest Path Properties

- **Upper-bound property** (**Ιδιότητα άνω φράγματος**)
  - We always have $d[v] \geq \delta(s, v)$ for all $v \in V$. Once $d[v]$ becomes equal to $\delta(s, v)$, it never changes again
- **Proof by induction**
  - Consider the relaxation of edge (u, v)
    - u already processed and (by inductive hypothesis) $d[u] \geq \delta(s, u)$
    - $d[v] = d[u] + w(u, v)$
      $\geq \delta(s, u) + w(u, v)$ (by inductive hypothesis)
      $\geq \delta(s, v)$ (by triangle inequality)
  - Once $d[v] = \delta(s, v)$, $d[v]$ will never change again
    - It cannot decrease, or it would violate $d[v] \geq \delta(s, v)$
    - It cannot increase, because Relaxation never increases d values

Shortest Path Properties

- **Convergence property** (**Ιδιότητα σύγκλισης**)
  - If $s \rightsquigarrow u \rightarrow v$ is a shortest path and $d[u] = \delta(s,u)$ before we relax edge (u,v), then $d[v] = \delta(s,v)$ afterward
- Proof:
  - After relaxing edge (u,v)
    - $d[v] \leq d[u] + w(u,v)$
      $= \delta(s,u) + w(u,v)$
      $= \delta(s,v)$ (optimal substructure)
  - By the upper-bound property
    - $d[v] \geq \delta(s,v)$
  - Therefore
    - $d[v] = \delta(s,v)$

Bellman-Ford Algorithm

```
BellmanFord(G, W, s)
   for each v ∈ G.V
      d[v] = ∞;
   d[s] = 0;
   for i = 1 to |G.V| - 1
      for each edge (u,v) ∈ G.E
         Relax(u, v, w);
   for each edge (u,v) ∈ G.E
      if (d[v] > d[u] + w(u,v))
           return FALSE;
   return TRUE;
```

Initialize d[], which will converge to shortest-path value δ

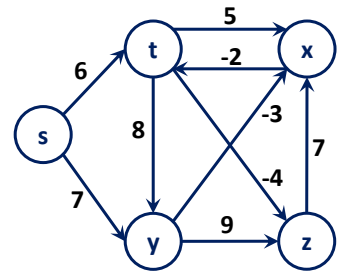
Relaxation: Make |V|-1 passes, relaxing each edge

Test for solution
Under what condition do we get a solution?

```
Relax(u,v,w): if (d[v]>d[u]+w(u,v)) then d[v]=d[u]+w(u,v)
```

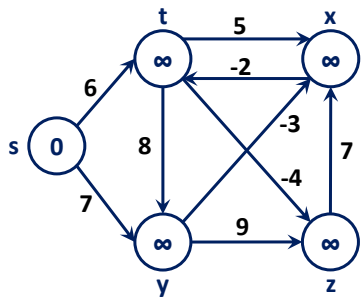
Bellman-Ford Algorithm

```
BellmanFord(G, W, s)
   for each v ∈ G.V
      d[v] = ∞;
   d[s] = 0;
   for i = 1 to |G.V| - 1
      for each edge (u,v) ∈ G.E
         Relax(u, v, w);
   for each edge (u,v) ∈ G.E
      if (d[v] > d[u] + w(u,v))
           return FALSE;
   return TRUE;
```

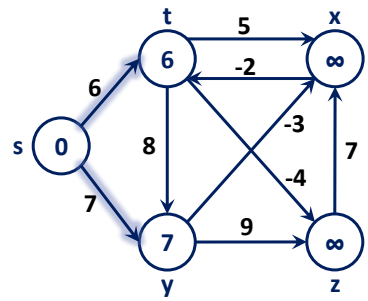



```
Relax(u,v,w): if (d[v]>d[u]+w(u,v)) then d[v]=d[u]+w(u,v)
```


Bellman-Ford Algorithm



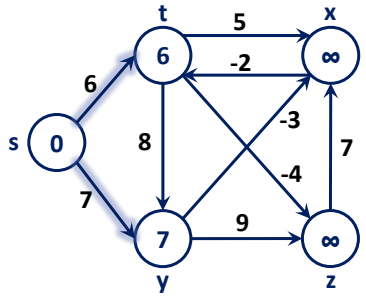
After initialization



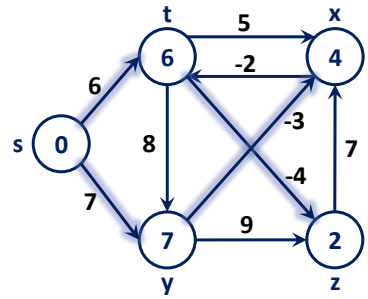
After 1st pass over edges

Edge ordering:
(t, x) (t, y) (t, z) (x, t) (y, x) (y, z) (z, x) (s, t) (s, y)

Bellman-Ford Algorithm



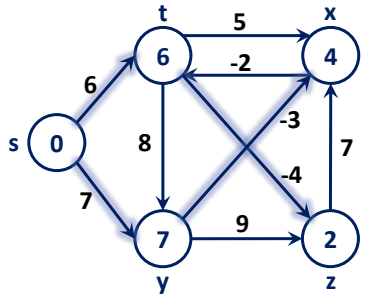
After 1st pass over edges



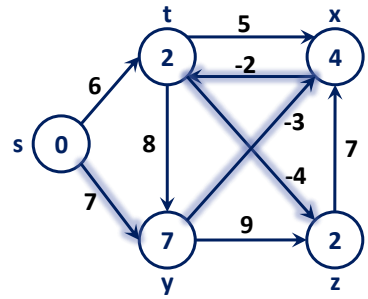
After 2nd pass over edges

Edge ordering:
(t, x) (t, y) (t, z) (x, t) (y, x) (y, z) (z, x) (s, t) (s, y)

Bellman-Ford Algorithm



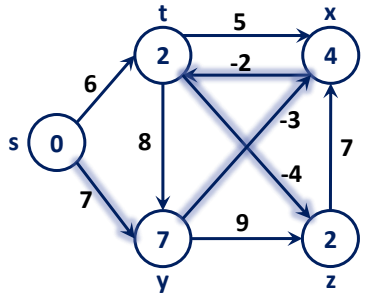
After 2nd pass over edges



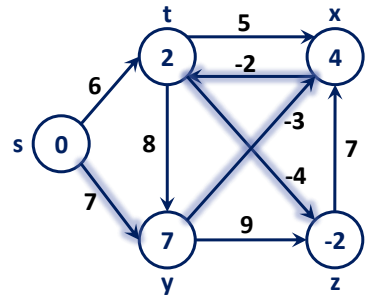
After 3rd pass over edges

Edge ordering:
(t, x) (t, y) (t, z) (x, t) (y, x) (y, z) (z, x) (s, t) (s, y)

Bellman-Ford Algorithm



After 3rd pass over edges



After 4th pass over edges

Edge ordering:
(t, x) (t, y) (t, z) (x, t) (y, x) (y, z) (z, x) (s, t) (s, y)

Bellman-Ford Algorithm

```
BellmanFord(G, W, s)
   for each v ∈ G.V
      d[v] = ∞;
   d[s] = 0;
   for i = 1 to |G.V| - 1
      for each edge (u,v) ∈ G.E
         Relax(u, v, w);
   for each edge (u,v) ∈ G.E
      if (d[v] > d[u] + w(u,v))
           return FALSE;
   return TRUE;

Relax(u,v,w): if (d[v]>d[u]+w(u,v)) then d[v]=d[u]+w(u,v)
```

What is the running time? $\Theta(VE)$

Bellman-Ford

- **Correctness:** if there are no negative-weight cycles, show $d[v] = \delta(s, v)$ after $|V| - 1$ passes
  - We know that $d[v] \geq \delta(s, v)$ (Upper-bound property)
  - Consider shortest path from s to v:

$$s \to v_1 \to v_2 \to \cdots \to v$$

    - Initially, $d[s] = 0$ is correct, and doesn't change (*Why?*)
    - After 1 pass through edges, $d[v_1]$ is correct (*Why?*) and doesn't change
    - After 2 passes, $d[v_2]$ is correct and doesn't change
    - …
    - Terminates in $|V| - 1$ passes (*Why?*)
    - *What if it doesn't?*

Bellman-Ford

- **Correctness (cont'd):** if there are negative-weight cycles, prove that Bellman-Ford returns FALSE
  - Consider neg-weight cycle $v_0 \to v_1 \to \cdots \to v_k$ where $v_0 = v_k$
  - Then, $\sum_{i=1}^{k} w(v_{i-1}, v_i) < 0$
  - Assume, Bellman-Ford returns TRUE
    - $d[v_i] \leq d[v_{i-1}] + w(v_{i-1}, v_i)$ for $i = 1,2, \ldots, k$
  - Summing the above for all $i = 1,2, \ldots, k$

$$\sum_{i=1}^{k} d[v_i] \leq \sum_{i=1}^{k} d[v_{i-1}] + \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

  - But $\sum_{i=1}^{k} d[v_i] = \sum_{i=1}^{k} d[v_{i-1}]$ (*Why?*)
  - $\Rightarrow \sum_{i=1}^{k} w(v_{i-1}, v_i) \geq 0$. Contradiction